



**UNIVERSIDAD
DE ANTIOQUIA**

1 8 0 3

Grafana - OpenFMB Integration

Juan Guillermo Calle Jiménez
Kevin David Martinez
Jhon Eduar Garcia
Colaboradores

Daniel Ospina Acero
Jaime Alberto Vergara Tejada
Supervisores

Sergio Armando Gutiérrez Betancur
Coordinator
saguti@ieee.org

June 11, 2026

Contents

1	Abstract	4
2	Acknowledgment	4
3	Acronyms	4
4	Project Structure	5
5	Architecture Overview	5
6	Top-Level Directory Structure	6
7	Backend — Rust (Server/)	6
8	Frontend — Angular (Client/)	7
9	Infrastructure	7
9.1	Docker Compose (docker-compose.yml)	7
9.2	OpenFMB Adapter (config_yaml/adapter.yaml)	8
9.3	Modbus Register Mapping (modbus-master/modbus-meter.yaml)	8
9.4	Database Schema (timescaledb.sql)	8
9.5	Users and Authentication	8
10	Equipment	9
11	Ubuntu	9
11.1	Executable	9
11.2	Development	10
12	Windows	11
12.1	Executable	11
12.2	Development	12
13	Introduction	13
14	Workflow	14
14.1	Developing	14
14.2	Final Product or Released Version	15
15	Files Modified	16
15.1	Auto Zoom and Institutional Images	16
15.2	Grafana Embedded Dashboard	17
15.3	General Notes	17
16	Docker	17
17	Grafana	19

18 HMI	19
18.1 Auto Zoom	19
18.1.1 Designer (<code>designer.component.ts</code>)	20
18.1.2 HMI (<code>hmi.component.ts</code>)	20
18.2 Adding Images to the HMI	21
18.2.1 Image Preparation	21
18.2.2 Toolbar Registration (<code>toolbar.json</code>)	21
18.2.3 Institutional Images Added	22
18.3 Widget Interconnection	22
18.3.1 Switches and Set Points	22
19 Grafana Integration — GrafanaDialogComponent	23
19.1 Architecture	23
19.2 Key Methods	23
19.2.1 <code>mapOpenFmbToPostgres(path)</code>	23
19.2.2 <code>buildSqlFromHmiData()</code>	23
19.2.3 <code>refreshIframe()</code>	24
19.3 Dashboard Configuration (<code>hmi-measure-variable.json</code>)	24
19.4 Compatibility Conventions	25
20 Evidences	26
20.1 Auto Zoom	26
20.2 Overflow	28
20.3 Grafana Plotting Window	30
20.4 Switches and Set-Points linking	32
20.5 Inserted Images	33
21 Docker Commands	34
22 Git Commands	35
23 Naming Convention	36
24 Software Versions	37
24.1 Ubuntu	37
24.2 Windows	38

1 Abstract

The *Universidad of Antioquia*, with the cleverness of the research groups *GIMEL*, as the leading group, and *GITA*, as support, started a combined project to develop a *Smart Room* for the *Sede Oriente de la Universidad de Antioquia*, to aggregate monitoring capabilities and sustainability.

In addition, innovative and up-to-date allied companies are involved in this project, and they are: Celsia, Haceb, PTI (Potencia y tecnologías incorporadas s.a.) and Promigas.

This document gathers the experience, software and main processes to incorporate plotting capabilities into an HMI by *OpenFMB* via the *Grafana Platform*. Here, the *Grafana Api* is the main asset for this goal. Also, changes to the interactivity, logic and connection between different widgets of the HMI are worked on.

The purpose of this document is to provide a well-detailed report, with every step taken, every software and their versions, etc. It can also be used as reference documentation.

2 Acknowledgment

We sincerely thanks the help and acknowledge the intelligence, hard work and productivity of the professionals involved in this combined project.

- Universidad de Antioquia
- Research group GIMEL and professor Nicolas Muñoz Galeano, GIMEL leader.
grupo.gimel@udea.edu.co, nicolas.munoz@udea.edu.co
- Research group GITA and professor Sergio Armando Gutiérrez Betancur, GITA coordinator.
gita.udea.edu.co, saguti@ieee.org
- Daniel Ospina Acero, GITA leader.
daniel.ospina3@udea.edu.co

And all intern students, outer professionals, MSc students from the research groups and all personal that helped, thank you so much.

In addition, we sincerely thank the allied companies.

3 Acronyms



Figure 1: Allied Companies and Research Groups

HMI	=	Human-Machine Interface
WSL	=	Windows Subsystem for Linux
OpenFMB	=	Open Field Message Bus
TCP	=	Transmission Control Protocol
CORS	=	Cross-Origin Resource Sharing
OS	=	Operative system
GITA	=	Grupo de Investigación en Telecomunicaciones Aplicadas
GIMEL	=	Grupo de Manejo Eficiente de la Energía

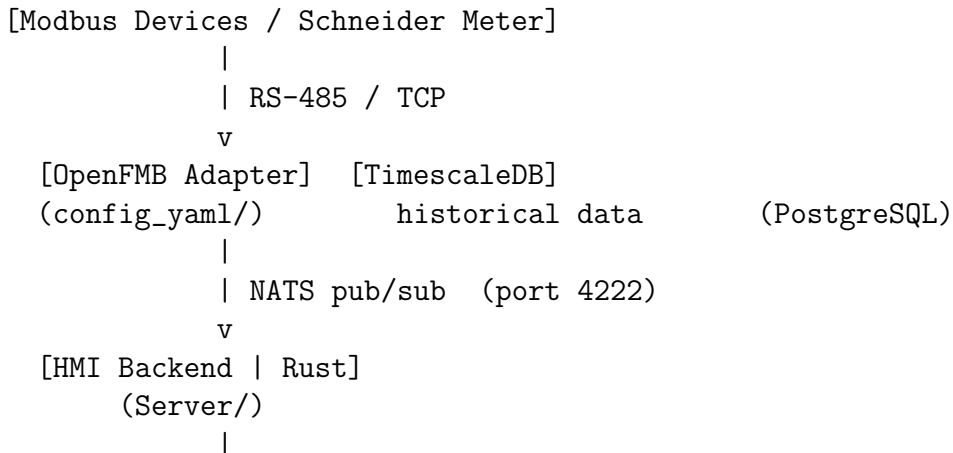
Table 1: Acronyms

4 Project Structure

This chapter provides a high-level overview of the **pp4-hmi-develop** repository, an OpenFMB HMI for the **PERSEO P4** project, jointly developed by the *Universidad de Antioquia (UDEA)* and *Open Energy Solutions Inc. (OESI)*.

5 Architecture Overview

The system is organized into four main communication layers:



```

      | WebSocket + HTTP  (port 32771)
      v
[HMI Frontend | Angular]
  (Client/)
    |
    | HTTP iframe  (port 3000)
    v
  [Grafana]
  (embedded dashboards)

```

Data flow:

1. The Schneider meter sends measurements over Modbus (RS-485/TCP).
2. The OpenFMB adapter polls those registers every second, converts them to `OpenFMB MeterReadingProfile` messages, and publishes them to NATS while simultaneously writing to TimescaleDB.
3. The Rust backend subscribes to NATS topics and forwards telemetry to the Angular frontend via WebSocket.
4. The frontend renders real-time data on interactive single-line diagrams (`mxGraph`) and embeds Grafana time-series panels via `iframe` for historical plotting.
5. Control commands (switch open/close, set-point adjustments) travel in the reverse direction: HMI → NATS → adapter → Modbus device.

6 Top-Level Directory Structure

```

pp4-hmi-develop/
Server/           # Backend | Rust (warp + tokio + riker actors)
Client/          # Frontend | Angular 15 (mxGraph + NgRx)
config/          # Runtime config: app.toml, users.json, diagrams/
config_yaml/     # OpenFMB adapter configuration (adapter.yaml)
modbus-master/   # Modbus register mapping (modbus-meter.yaml)
scripts/        # Python utilities: NATS subscriber, Modbus slave
Dockerfile       # Multi-stage build: Rust + Node.js → Alpine image
docker-compose.yml # Service orchestration (NATS, adapter, DB, HMI)
timescaledb.sql  # TimescaleDB schema (hypertables)
requirements.txt  # Python dependencies (FastAPI, pymodbus)

```

7 Backend — Rust (Server/)

An asynchronous web server built with **warp** (HTTP framework) and **tokio** (async runtime). It uses a **riker** actor model to coordinate message passing between the NATS bus and the frontend WebSocket connections. Authentication is handled via JWT with bcrypt password verification.

Key files under `Server/src/`:

- `bin/hmi_server.rs` — Entry point: initializes warp routes, logging, and the actor system.

- `auth.rs` — JWT issuance and bcrypt credential verification.
- `handler.rs` — HTTP route handlers: login, diagram CRUD, WebSocket upgrade.
- `hmi/coordinator.rs` — Coordinates messages between actors (subscriber, publisher, processor).
- `hmi/hmi_subscriber.rs` — Subscribes to OpenFMB NATS topics and dispatches incoming telemetry.
- `hmi/hmi_publisher.rs` — Publishes control commands back to NATS.
- `hmi/processor.rs` — Transforms OpenFMB protobuf messages into HMI-consumable structures.

8 Frontend — Angular (Client/)

A Single Page Application (SPA) built with **Angular 15**, **Angular Material 14**, and **NgRx 12** for reactive state management. **mxGraph** is used as the engine for creating and rendering single-line electrical diagrams.

Key modules under `Client/src/app/`:

- `designer/` — Diagram editor: drag-and-drop mxGraph canvas for building single-line diagrams with electrical components.
- `hmi/` — Live HMI view: real-time data overlay on diagrams, device control dialogs (switches, regulators, set-points), and embedded Grafana panels.
- `store/` — NgRx state management: actions, effects, and reducers for `auth`, `designer`, and `hmi` modules.
- `shared/` — Reusable components (sidebar, breadcrumbs, notifications), services (diagrams, navigation, themes), directives, and data models.
- `assets/images/toolbar/` — SVG icons for the designer toolbar: standard electrical components plus institutional imagery.
- `assets/json/toolbar.json` — Configuration file defining all draggable elements in the designer toolbar.

9 Infrastructure

9.1 Docker Compose (`docker-compose.yml`)

Four services run in a shared `openfmb_dev` Docker network:

- `nats` (`nats:alpine`, port 4222) — NATS message broker; the communication backbone for all OpenFMB messages.
- `adapter` (`oesinc/openfmb.adapters`) — Reads Modbus registers, converts them to OpenFMB profiles, publishes to NATS, and writes structured data to TimescaleDB.

- **timescaledb** (`timescale/timescaledb:latest-pg14`, port 5432) — PostgreSQL-based time-series database for historical electrical measurements.
- **openfmb-hmi** (`openfmb-hmi:latest`, port 32771) — The HMI server combining the compiled Rust backend and the Angular frontend.

9.2 OpenFMB Adapter (`config.yaml/adapter.yaml`)

Configures three adapter plugins working in concert: **modbus-master** polls the physical device at 1-second intervals, **nats** publishes OpenFMB profiles to the broker, and **timescaledb** persists structured data and raw JSON messages for historical querying.

9.3 Modbus Register Mapping (`modbus-master/modbus-meter.yaml`)

Maps Modbus holding registers from a Schneider meter to OpenFMB **MeterReadingProfile** fields. Covers three-phase voltage (PhV, PPV), current (A), active power (W), reactive power (VAr), apparent power (VA), power factor (PF), frequency (Hz), and energy accumulators (TotWh, TotVAh, etc.).

9.4 Database Schema (`timescaledb.sql`)

Two **hypertables** partitioned in 1-hour chunks:

- **data** — Structured table with 40+ columns covering magnitudes and angles for all three phases. Indexed by **device_uuid** (clustered) and **timestamp**.
- **raw_data** — Semi-structured table storing JSONB values by **tagname**, allowing flexible queries without schema changes.

9.5 Users and Authentication

Three predefined users stored in `config/users.json` (passwords hashed with bcrypt):

- **admin** — Full system access. Also used by the Grafana API for automated operations.
- **engineer** — Design and configuration access.
- **demo** — Viewer only.

The backend issues JWT tokens upon login. The Grafana integration requires credentials **admin** / **hmiadmin** to be consistent between the HMI and Grafana.

10 Equipment

The following equipment was used to test all the documented processes and features described in this work.

- Asus VivoBook S, Modelo S5101U, 12 GB DDR4 RAM, Intel core I5-8250U. WSL with Ubuntu distro. SSD. Windows 10 OS.
- Acer E1-472 6496, 8 GB DDR3 RAM, 8 GB swap, SSD. Lubuntu distro, Linux OS. Intel Core I5 4200U.

11 Ubuntu

This section shows the software and tools to install to develop or run the final user product.

See the software versions, in the Appendix part for more information.

11.1 Executable

This section is about the minimum software and resources for running the final user product.

1. Download and configure the **Docker Desktop**. Follow this link: Install Docker Desktop on Ubuntu and see the extended Docker section for more info.
2. **ModbusPal** was used in the project to emulate devices with Modbus protocols. Install any Java JDK to run this software. For this project, **Coretto openjdk** was selected.
3. Download the whole project from any secure source. Check credentials for access. This project should have containers for: HMI, Grafana, timescale, reverse proxy, NATS, etc.
4. Update to the last **stable version uploaded** of the **HMI** image on dockerhub in your docker-compose.yml file.
5. Open the folder of the whole project, and from the root path, run the next command on the Terminal:

```
$ sudo docker compose up
```
6. Open a web browser (Chrome, Edge, Mozilla -preferable- or Opera are recommended) and type in the search bar: `http://localhost:3000`. This will open the **Grafana Platform**. Go to settings and change the credentials as user to: `admin` and password to: `hm1admin`.
7. Connect all devices or emulators to the network.
8. In the same or other browser tab, type in the search bar `http://localhost:32771`. This will open the HMI interface.

Done.

11.2 Development

This section is about the necessary software and tools for developing the HMI.

- Download **Inkscape**. This is for converting the image format into svg format.
- Download and configure the **Docker Desktop**. Follow this link: [Install Docker Desktop on Ubuntu](#) and see the extended Docker section for more info.
- Download the **HMI repository** and extract it into a new folder. Alternatively, download the modified version from any secure source. This repository is where all developments will take place.
link for original opensource files: <https://github.com/openenergysolutions/openfmb.hmi>.

- In order to compile the files, some programs (running as commands) need to be install. Check if they are installed, else run these commands on the Terminal from root path of this new folder.

1. To install node.

```
$ sudo apt install nodejs
```
2. To install npm.

```
$ sudo apt install npm
```
3. To install cmake.

```
$ sudo apt-get install cmake
```

Check that the installed version is lower than 3.5.
4. To install angular.

```
$ npm install -g @angular/cli
```
5. To install rust.

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```
6. Now, for the *first time*, from the project repository:
And, from the \Client folder, run the following command on the Terminal:

```
$ npm install --legacy-peer-deps  
$ npm run build  
$ cd ..  
$ cargo run
```

This will build the **debug files** for the front-end and back-end. Run `$ npm run build` on the Terminal every time you want to compile the changes of the front-end (i.e., changes made in the /Client folder); For changes in the back-end (i.e., changes in the /Server folder), run `$ cargo run`. The longest building time is for the first time these commands are run.

- Follow the same steps as in the previous section to test all component together, with the exception that the HMI image configured in the docker-compose.yml should be omitted, because this will be the one run from the debug code.
- To create the **released image**, run the following command on the Terminal from the root path of the modified HMI project: `$ docker build -t <image_name>:<tag>`.

To test a new image (which executes the same as tested), just follow the same steps as in the previous section.

12 Windows

This section shows the software and tools to install to develop or run the final user product.

See the software versions, in the Appendix part for more information.

12.1 Executable

This section is about the minimum software and resources for running the final user product.

1. Download and configure the **Docker Desktop**. Follow this link: [Install Docker Desktop on Windows](#) and see the extended Docker section for more info. Pay close attention to the installation of the WSL.
 - (a) Check system requirements
 - (b) Install **WSL**. FIRST !! IMPORTANT
 - (c) Restart computer
 - (d) Run Linux WSL first
 - (e) Install Docker Desktop

The following is the version info for the WSL:

```
WSL version: 2.4.13.0
Kernel version: 5.15.167.4-1
WSLg version: 1.0.65
MSRDC version: 1.2.5716
Direct3D version: 1.611.1-81528511
DXCore version: 10.0.26100.1-240331-1435.ge-release
Windows version: 10.0.19045.5737
```

2. **ModbusPal** was used in the project to emulate devices with Modbus protocols. Install any Java JDK to run this software. For this project, **Coretto openjdk** was selected.
3. Download the whole project from any secure source. Check credentials for access. This project should have containers for: HMI, Grafana, timescale, reverse proxy, NATS, etc.
4. Update to the last **stable version uploaded** of the **HMI** image on dockerhub in your docker-compose.yml file.
5. Open the folder of the whole project, and from the root path, run the following commands on the Command Prompt:

```
$ sudo docker compose up
```
6. Open a web browser (Chrome, Edge, Mozilla -preferable- or Opera are recommended) and type in the search bar: `http://localhost:3000`. This will open the Grafana Platform. Go to settings and change the credentials as user to: `admin` and password to: `hmladmin`.
7. Connect all devices or emulators to the network.
8. In the same or other browser tab, type in the search bar `http://localhost:32771`. This will open the HMI interface.

Done.

12.2 Development

This section is about the necessary software and tools for developing the HMI.

- Download **Inkscape**. This is for converting the image format into svg format.
- Install `Node.js` and `npm` (version 10.9.2):
 1. Install Python.
 2. Install the **Visual Studio Build Tools 2022** tool. Follow the next link for more information: <https://github.com/bycloudai/InstallVSBuildToolsWindows>
 3. Follow this link, download the .msi installer and run: <https://nodejs.org/en/download/>

- Install **angular** as

```
$ npm install -g @angular/cli
```

- Install **Rust**. Download the 64/32 bit version for windows from:

```
https://www.rust-lang.org/tools/install
```

Check if you already have "Visual Studio C++ Build tools", as in the previous step. If not, the installer will prompt you to install it.

- Install `cmake`:

```
https://cmake.org/download/
```

Download a lower version than 3.5 or check if any higher version works.

- Now, for the *first time*, from the project repository:

And, from the Client folder, run the following commands on the Command Prompt:

```
$ npm install --legacy-peer-deps
```

```
$ npm run build
```

```
$ cd ..
```

```
$ cargo run
```

This will build the **debug files** for the front-end and back-end. Run `$ npm run build` on the Command Prompt every time you want to compile the changes of the front-end (i.e., changes made in the /Client folder); For changes in the back-end (i.e., changes in the /Server folder), run `$ cargo run`. The longest building time is for the first time these commands are run.

- Follow the same steps as in the previous section to test all components together, with the exception that the HMI image configured in the `docker-compose.yml` should be omitted, because this will be the one run from the debug code.
- To create the **released image**, run the following command on the Command Prompt from the root path of the modified HMI project: `$ docker build -t <image_name>:<tag>`.

To test a new image (which executes the same as tested), just follow the same steps as in the previous section.

13 Introduction

The purpose of the project is to incorporate plotting capabilities into the HMI by OpenFMB via the Grafana Platform. Here, the Grafana Api is the main asset for this goal. Also, changes to the interactivity, logic and connection between different widgets of the HMI are worked on.

It is important to note that the HMI must have autonomy and management over the Grafana Platform, so user intervention is necessary just for the very essential configurations, such as credential settings.

At the moment of writing this document, the HMI is unable to create a new data source in the Grafana Platform, so at least one must be created a prior.

In addition, many tools, platforms, or software were used in the project. Among them, the most relevant are briefly explained:

- **OpenFMB**

The OpenFMB is a grid-edge interoperability standard that specifies common data models and unified protocol interfaces to democratize the utility ecosystem and enable distributed intelligence use-cases. OpenFMB reduces the need for centralized control and allows management of distribution systems at the circuit level.

- **Hmi and its open-source code**

One of the tools provided by OpenFMB is the HMI, which is an out-of-the-box demo interface to communicate with devices connected to a network running the OpenFMB protocol. This enables the user to visualize data and control devices previously configured. However, even though this HMI is very practical, it could have limited capabilities for more complex functions, such as plots to display data. In addition, users might like to have ownership over the product they design. Here, OpenFMB lets them to download their open-source code to make changes, add images and content to appropriate their designs.

- **Grafana**

Grafana is a multi-platform open source analytics and interactive visualization web application. It can produce charts, graphs, and alerts for the web when connected to supported data sources.

- **Grafana Api**

The Grafana backend exposes an HTTP API, which is the same API that's used by the frontend to do everything from saving dashboards, creating users, and updating data sources.

All the configurations request by the HMI are performed by this Api.

- **Docker**

Docker helps developers bring their ideas to life by conquering the complexity of app development. Actively used by millions of developers around the world, Docker Desktop and Docker Hub provide unmatched simplicity, agility and choice.

It can be stated that with Docker you "containerize" your application into images, enabling any user to run it on a Docker environment (context) with other so-called images. When a new version is released for any image, only this one could be changed, whereas the others are left untouched.

Now, when developing the project, some interesting concepts, problems, troubleshooting and suggestions came up, and they are exposed in the following sections.

See the Appendix part for complementary information.

14 Workflow

There are common workflows when developing the project or running the released version. See the Setup part for instructions on setting up and first steps.

First, developing is about: HMI, reverse proxy and Grafana API.

Second, the release version is about: docker containers, docker images, extra files and folders for running everything properly. Note that some configurations and tools, such as the timescale database, are not explained in this document. this information should be found in other reports or documents, so if needed, contact someone in charge of the project. This may be included in future updates of this document.

14.1 Developing

Now, when developing, the common workflow goes as follows:

1. Download the project. Download it from the openFMB website if starting from scratch or ask for a repository with a modified version.
2. Start Docker Desktop or the Docker Engine.
3. Open your preferred text editor or IDE.
4. Make sure to have a version control like GIT and implement it in the project.
5. Unzip / extract file and open the folder of the project.
6. Here, some initial files and folders will be found and others will be added when creating the front-end and back-end for the first time.

This development stage is very practical and allows the developer to run the HMI with a testing interface, without creating a new docker image every time, which takes much longer. This testing interface is not meant to be use as the final released product because it not as portable and easy to incorporate like a docker image.

7. To create the front-end for the first time, go to Client/ and run the following commands in the terminal from the folder path:

```
$ npm install --legacy-peer-deps
$ npm run build
```

8. From now on, to update the front-end run the following command from the Client/ folder:

```
$ npm run build
```

9. To create the back-end files, return to the first level of the project folder and run the next command in the terminal from it:

```
$ cargo run
```

This command will run the testing interface, as well, but wouldn't recreate/recompile any file if the back-end was not modified.

10. Hence, after creating the front-end and back-end for the first time, some relevant files and folder are:

- **users.json file:** Registered users on the HMI. Managed from the HMI site.
 - **equipment.json file:** Information to simplify the usage and recognition of a MRID by using an alias. Managed from the HMI site.
 - **Dockerfile file:** File to create the release or docker image.
 - **Client folder:** This is where the Front-end takes form.
 - **Server folder:** This is where the Back-end takes form.
 - **config folder:** Folder that saves the basic configuration for the HMI site.
 - **target:** Back-end temporary files created when running the "cargo run" command.
 - **Client/dist:** Front-end temporary files created when running the "npm run build" command.
 - **Client/src:** Source Files, like images, html, style, logic files, for the front-end development.
11. Everything is done. Open the testing interface on a web browser by typing localhost:32771. Also, by default, the HMI connects to port :4222 to read NATS messages.
 12. Once all changes have been made and the release version is ready to be created, run the following command in the terminal from the first level of the project folder:

```
$ docker build -t <image_name>:<tag>.
```

Note: The idea is to integrate Grafana plotting capabilities within the HMI. Nevertheless, the HMI itself reads OpenFMB-based messages, which means some devices may be running and communicating. This requires some configuration and other entities running to put everything together, like adapters, emulators, servers, etc. So, to test with practical and on-site reading it is necessary to include and communicate with this entities while testing. For this project, all of this is joint-together in a docker-compose.yml file, both for the development and the release version, with the exception of running a provisional emulator (like ModbusPal) when no devices are connected and active to the network. Hence, the .yml file must have an HMI section/property and this one will be enabled when running the final/released product and disabled when testing, whereas the other containers are left untouched. This helps the productivity and, again, to verify the proper functionality of the development.

14.2 Final Product or Released Version

When running the final product or released version, the common workflow goes as follows:

1. Make sure that the last image created or uploaded to the cloud is included along with the must-have commands in the docker-compose.yml file.
2. Start Docker Desktop or the Docker Engine. Additionally, run the WSL before when the OS is Windows.
3. Verify and change, if needed, the static IP address in the configuration file for the HMI adapter (/config/adapters.yaml file).
4. When no physical devices are connected or some are missing, use ModbusPal to add Modbus-based devices. This document only considers devices running under the Modbus protocol.
5. Run the whole project/programs from the .sh file.

6. It is recommended to find a way to auto-start the Docker Engine and to run the .sh file, for a better user experience.
7. For information on how to add images, configure the .yml file or how to use any of the added features, refer to the specific topic in this document.
8. Open one web browser and go to the HMI site or the Grafana site with localhost:32771 or localhost:3000, respectively.
9. When going to the HMI site, enter your credential. When logging into for the first time, use "admin" as user and "hmladmin" as password. Do not delete these default credentials because they are used by the Grafana API to authenticate, instead create a new one.
10. When going to the Grafana site, enter your credentials. When logging into for the first time, use "admin" for user and password, then create a new user (or change the default one) with all privileges and set the user as "admin" and password as "hmladmin".
11. That's all.

At the moment of writing this document, no information on configuring the openFMB protocol is added. For this specific type of information, please refer to the openFMB official documentation or ask someone in charge for documentation regarding the project on this topic.

15 Files Modified

This section lists the most relevant files modified or created across the two feature branches. Further details are covered in their respective sections of this document.

15.1 Auto Zoom and Institutional Images

- `Client/src/app/designer/designer.component.ts`
Added the `applyAutoZoom()` method: calls `graph.fit(0, false)` and `graph.center(true, true)` after the diagram XML is loaded. See Section 18.1.
- `Client/src/app/designer/designer.component.scss`
Toolbar width changed from a fixed 145px to a responsive range: `min-width: 150px; max-width: 190px`. This accommodates larger institutional images without overflow.
- `Client/src/app/hmi/hmi.component.ts`
Added the `applyAutoZoom()` method: calls `graph.fit(20, false)`, `graph.center(true, false)`, and `graph.setPanning(true)` after the diagram loads. See Section 18.1.
- `Client/src/assets/json/toolbar.json`
27 new entries of type "UdeA" registered for the institutional SVG images.
- `Client/src/assets/images/toolbar/UdeA_*.svg`
54 SVG files (27 images $\times$ 2: original + -copy duplicate) covering the visual identity of: Universidad de Antioquia, GIMEL, GITA, Haceb, CELSIA, Promigas, PTI, IUPB, Universidad del Chocó, and the PERSEO project.
- `config/diagrams/03eadab4-...json` and `3639be96-...json`
Two saved diagrams updated to include the new institutional visual elements.

15.2 Grafana Embedded Dashboard

- `Client/src/app/hmi/dialogs/grafana-dialog/`
New Angular dialog component (`.ts`, `.html`, `.scss`) implementing the Grafana iframe integration with automatic SQL generation. See Section on Grafana API Integration.
- `config/grafana/dashboards/hmi-measure-variable.json`
Dashboard updated to schema version 40, `${sql:raw}` template variable, 10-second auto-refresh, and `now-15m` default time range.
- `modbus-master/modbus-meter.yaml`
Seven register mappings activated (previously set as ignored):

OpenFMB Variable	Description	Modbus Registers
<code>W.phsA.mag</code>	Active power, phase A	3010 / 3009
<code>PPV.phsAB.mag</code>	Line voltage AB	3036 / 3035
<code>PPV.phsBC.mag</code>	Line voltage BC	3060 / 3059
<code>PPV.phsCA.mag</code>	Line voltage CA	3068 / 3067
<code>PPV.phsAB.ang</code>	Voltage angle AB	3076 / 3075
<code>VA.phsA.mag</code>	Apparent power, phase A	3084 / 3083
<code>VAr.phsA.mag</code>	Reactive power, phase A	3110 / 3109

15.3 General Notes

- **Rust version:** This project uses **rustc 1.90.0**. It is recommended to match the local Rust installation with the version in the Dockerfile (`rust:1.90.0-alpine3.21`). Download from: <https://rust-lang.org/tools/install/>
- `cargo.lock`: The lockfile must remain tracked in version control. Removing or regenerating it can cause dependency inconsistencies when building Docker images on different machines. Leave it as provided and only update specific library versions when necessary.
- **Constant conditionals:** Avoid code patterns where a compile-time constant boolean guards a block, such as:

```
<outer-statement> { if (this.debug) { <code> } }
```

When `this.debug` is a constant `false`, the Angular compiler eliminates the inner block entirely, turning the outer statement into an empty body. This can cause runtime errors that are difficult to trace. Use environment flags or runtime configuration instead.

16 Docker

The changes regarding Docker are described in this section:

- `docker compose up != docker-compose up`
This is because `docker-compose` is the V1 syntax, which is now deprecated. The V2 syntax is `docker compose`, since it's now a plugin and not a separate command.

- Follow this steps to sign into docker desktop:

<https://docs.docker.com/desktop/setup/sign-in/>

- To see the docker containers in the docker desktop application is necessary to create a context for a normal user (no `sudo`) and the superuser (with `sudo`), link them, and set these contexts as the current contexts for both cases.

Steps:

1. Create the context for the normal user. By default, docker desktop creates a context named *desktop-linux*. If not created by default, try running the command:

```
$ docker context
```

If doesn't work, search how to activate the default context or reinstall docker desktop.

This default context can be used to observe the running containers, or a new one can be created. The default context of docker desktop was selected for the project.

To create a new context, run the command:

```
$ docker context create --docker <path> <name>
```

<path> should lead to a `.sock` file.

<name> is the context name. Feel free to put any name

2. Check the available contexts with the following command:

```
$ docker context ls
```

The context with the `*` is the active one. If the current active context is not the desired one, select another context with:

```
$ docker context use <context name>
```

3. Create and link a context for the superser. A new context should be created with the same path as the one created or used by the normal user. For example, if you run the command:

```
$ docker context ls
```

You may get the desktop-linux context as:

```
desktop-linux * Docker Desktop unix:///home/<user>/.docker/desktop/docker.sock
```

Here, copy the path (`unix:///home/<user>/.docker/desktop/docker.sock`) and save it for later. Now, check the current context for the superuser as:

```
$ sudo docker context ls
```

If the active context is the same (same path) as the one used for the the normal user, no change is needed. However, if they are different (different path), try changing the current context for one of the available options with the command:

```
$ sudo docker context use <context name>
```

If no available context has the same path as the one selected for the normal user, create a new one with:

```
$ sudo docker context create --docker <path> <name>
```

<path> the same as in the context selected for the normal user

<name> ideally, the same name as the context selected for the normal user

Thus, if the context desktop-linux is not created, create it with:

```
$ sudo docker context create --docker unix:///home/<user>/.docker/desktop/docker.sock
```

Now, select the created context as the active context for the superuser.

It's done.

4. The context `desktop-linux` only runs when docker desktop is open/started/active. The containers will not run if it is quit it is not started yet
- Run docker (.yml) files as superuser or administrator. This is because most of the images or emulators need to open TCP ports and require special permissions.

17 Grafana

The configuration changes required for Grafana to work with the HMI are described in this section.

- Use a specific pinned version of Grafana rather than `:latest`, to ensure reproducibility. This project uses version **11.5.1**.
- To inspect the current Grafana runtime settings, navigate to: **Home → Administration → General → Set**
- To allow Grafana panels to be rendered inside iframes (required for the embedded dashboard feature), add the following environment variables to the Grafana service in `docker-compose.yml`:

`environment:`

- `GF_SECURITY_ALLOW_EMBEDDING=true`
 - `GF_AUTH_ANONYMOUS_ENABLED=true`
 - `GF_AUTH_ANONYMOUS_ORG_NAME=UdeA`
 - `GF_AUTH_ANONYMOUS_ORG_ROLE=Viewer`

 - `GF_SECURITY_ALLOW_EMBEDDING=true` — Permits Grafana to be embedded inside an iframe from an external origin.
 - `GF_AUTH_ANONYMOUS_ENABLED=true` — Removes the login requirement for anonymous viewers, so the iframe renders the panel without prompting for credentials.
 - `GF_AUTH_ANONYMOUS_ORG_NAME` and `GF_AUTH_ANONYMOUS_ORG_ROLE` — Define the organization and permission level for anonymous sessions.
- For Grafana API calls using HTTP Basic Auth (`user:password@host`), ensure Basic Auth is enabled in Grafana. It is enabled by default.
 - **CORS:** In the current stack, the HMI service exposes its port directly (`32771:32771`) and communicates with Grafana at `http://localhost:3000`. The **reverse-proxy** (nginx) service has been removed from `docker-compose.yml` and no additional CORS proxy is required for normal operation.

18 HMI

18.1 Auto Zoom

When a diagram is loaded, it is automatically scaled and centered in the viewport. This is handled by the `applyAutoZoom()` method, which is called inside the `finally` block of the diagram loading

routine — after `graph.getModel().endUpdate()` — so it always runs even when XML parsing errors occur.

The method behaves slightly differently in the designer and in the live HMI view:

18.1.1 Designer (`designer.component.ts`)

```
private applyAutoZoom() {
  if (this.graph) {
    this.graph.fit(0, false);      // Fit with 0 px border
    this.graph.center(true, true); // Center horizontally and vertically
  }
}
```

Listing 1: `applyAutoZoom` — `designer.component.ts`

- `fit(0, false)` — Scales the diagram to fill the full viewport with no surrounding margin.
- `center(true, true)` — Centers the content both horizontally and vertically.

The designer toolbar width was also adjusted to accommodate larger institutional images without visual overflow:

```
/* Before */
width: 145px;

/* After */
min-width: 150px;
max-width: 190px;
```

Listing 2: `designer.component.scss` — toolbar width

18.1.2 HMI (`hmi.component.ts`)

```
private applyAutoZoom() {
  if (this.graph) {
    this.graph.fit(20, false);      // Fit with 20 px border
    this.graph.center(true, false); // Center horizontally only
  }
}
```

Listing 3: `applyAutoZoom` — `hmi.component.ts`

- `fit(20, false)` — Leaves a 20-pixel border around the diagram for a more comfortable visual margin.
- `center(true, false)` — Centers horizontally but leaves the vertical axis free for scrolling, which is natural for tall electrical diagrams.
- `this.graph.setPanning(true)` — Panning is now enabled so the user can drag to navigate the diagram after it loads.

18.2 Adding Images to the HMI

Custom images can be added to the designer toolbar as draggable elements. The procedure is the same regardless of whether the project is running in development or released (Docker) mode, since the images are copied into the container at startup via the Docker endpoint.

18.2.1 Image Preparation

1. Convert the image to SVG format. **Inkscape** is the recommended tool.
2. Rename the file with the prefix "UdeA_", for example: `UdeA_Logo.svg`.
3. Create a copy with the suffix `-copy`: `UdeA_Logo-copy.svg`. This duplicate is required by mx-Graph's toolbar rendering.
4. Place both files in `Client/src/assets/images/toolbar/`.

18.2.2 Toolbar Registration (toolbar.json)

Register the image in `Client/src/assets/json/toolbar.json` by adding a new entry in this format:

```
{
  "shape": "image",
  "image": "assets/images/toolbar/UdeA_Logo.svg",
  "points": [ [1, 0.5], [0, 0.5], [0.5, 0], [0.5, 1] ],
  "width": 100,
  "height": 100,
  "title": "UdeA Logo",
  "left": 0,
  "top": 0,
  "type": "UdeA"
}
```

Listing 4: toolbar.json — entry format

- `shape: "image"` — Tells mxGraph to render this item as an image element on the canvas.
- `image` — Relative path from the compiled `dist/` folder. Keep all custom images inside `assets/images/toolbar/` so the path resolves correctly after the Angular build.
- `points` — Connection anchor points on the four sides (right, left, top, bottom).
- `width / height` — Default size in pixels when the element is dragged onto the canvas.
- `title` — Label displayed in the sidebar of the designer.
- `type: "UdeA"` — Groups all institutional images under the same toolbar category.

Recommendation: Rather than placing many individual logo images onto a diagram, design the complete interface layout as a single background SVG (`UdeA_Interfaz-01.svg`) and send it to the back layer. This prevents it from overlaying or blocking interaction with the HMI widgets on top.

18.2.3 Institutional Images Added

As part of the PERSEO P4 visual identity, 27 SVG images were added to the toolbar (54 files total, counting the required -copy duplicates):

- **Universidad de Antioquia (UdeA)** — Full logo (color and B&W), logo with year, symbol, name text, and Microrred Sede Oriente label.
- **Research groups** — GIMEL and GITA logos.
- **Allied companies** — Haceb (3 variants: white-green, green, B&W), CELSIA, Promigas, PTI (Potencia y Tecnologías Incorporadas).
- **Partner universities** — IUPB (Inst. Univ. Pascual Bravo), Universidad del Chocó.
- **Project identity** — PERSEO project name, decoration elements, and a full-interface background image (UdeA_Interfaz-01.svg, 3000×1600 px).

18.3 Widget Interconnection

18.3.1 Switches and Set Points

Some devices use a protection register to block other registers from being modified. Switches that enable or disable set-point buttons must follow this MRID convention:

If the switch MRID is:

00000000-0000-0000-0000-0000000040247

Then the controlled set-point button MRID should be:

00000000-0000-0000-0000-004024740333

The general format is:

xxxxxxx-xxxx-xxxx-xxxx-xxYYYYZZZZZ

YYYYY = last 5 digits of the switch MRID

ZZZZZ = last 5 digits of the set-point MRID



Figure 2: Diagrams-HMI naming

19 Grafana Integration — GrafanaDialogComponent

The Grafana integration embeds live Grafana panels inside the HMI using Angular's dialog system. When the user right-clicks on a measurement widget and selects a variable to plot, a dialog opens with an iframe rendering the corresponding Grafana time-series panel. The SQL query, the device MRID, and the variable path are all resolved automatically — no manual configuration is required from the user.

The component is located at: `Client/src/app/hmi/dialogs/grafana-dialog/`

19.1 Architecture

HMI Diagram (mxGraph)

```
|
| right-click on measure widget → select variable
v
```

GrafanaDialogComponent (.ts / .html / .scss)

```
|-- mapOpenFmbToPostgres() → column name in TimescaleDB
|-- buildSqlFromHmiData() → parametrized SQL query
|-- refreshIframe() → sanitized Grafana URL
|
| HTTP GET :3000/d-solo/...?var-sql=<query>
v
```

Grafana Panel (iframe, /d-solo/ endpoint)

```
|
| SQL via ${sql:raw} template variable
v
```

TimescaleDB · table: data

(device_uuid, timestamp, <measurement column>)

19.2 Key Methods

19.2.1 mapOpenFmbToPostgres(path)

Translates an OpenFMB variable path into the corresponding column name of the `data` table in TimescaleDB.

- **Input:** `readingMMXU.PPV.phsAB.mag`
- **Process:** Splits the path by dots, locates the `readingMMXU` segment, takes the next two segments (`PPV`, `phsAB`), appends the measurement type (`mag` or `ang`), and lowercases the whole string.
- **Output:** `ppv_phsab_mag`

19.2.2 buildSqlFromHmiData()

Constructs the parameterized SQL query using the resolved column name and the device MRID:

```
SELECT * FROM (
  SELECT "timestamp" AS "time",
         <column>     AS "<alias>"
  FROM   data
```

```

WHERE device_uuid = '<mrid>'
ORDER BY "timestamp" DESC LIMIT 200
) AS subquery
ORDER BY "time" ASC

```

Listing 5: Auto-generated SQL query

The inner query retrieves the 200 most recent rows for the target device; the outer query sorts them chronologically for correct rendering in Grafana's time-series panel. The alias is taken from the dialog title (the human-readable variable name).

19.2.3 refreshIframe()

Builds the final Grafana panel URL and assigns it to the iframe via Angular's `DomSanitizer.bypassSecurityTr`

```

http://<host>:3000/d-solo/hmi-measure-variable/measure-box-variable
?orgId=1
&panelId=1
&theme=light
&from=now-15m
&to=now
&var-sql=<encoded_query>

```

Listing 6: Grafana panel URL structure

- `/d-solo/` — Renders only the panel, without Grafana's navigation bar or dashboard header.
- `var-sql` — Passes the SQL query as the `${sql:raw}` template variable defined in the dashboard.
- `from=now-15m` — The default time window shows the last 15 minutes, focused on recent data (reduced from 6 hours in earlier versions).
- The SQL textarea in the dialog is **read-only**; the query is generated automatically and is not manually editable by the user.

19.3 Dashboard Configuration (hmi-measure-variable.json)

The `hmi-measure-variable` Grafana dashboard was updated to match the new integration:

- **Schema version:** 39 → 40 (compatible with Grafana 11.5.1).
- **Panel type:** `timeseries` backed by a PostgreSQL/TimescaleDB datasource.
- **Template variable** `sql`: type `textbox`, receives the query from the frontend via `${sql:raw}`.
- **Auto-refresh:** `"refresh": "10s"` — the panel polls for new data every 10 seconds.
- **Default time range:** `now-15m` to `now`.
- **Panel title:** left blank so it can be controlled dynamically from the frontend.

19.4 Compatibility Conventions

- The **title** of each Grafana dashboard is automatically linked to the device MRID by the HMI when a new dashboard is created.
- The **description** of each panel is automatically linked to the OpenFMB variable path.
- Dashboards not related to this project must be named with the prefix "p_" to prevent the HMI logic from picking them up.
- The HMI and Grafana must share the same credentials: **user** admin, **password** hm1admin. Do not remove the default **admin** account — it is used internally by the Grafana API.

20 Evidences

In this part, now and then are compared to show the progress done.

20.1 Auto Zoom

Then

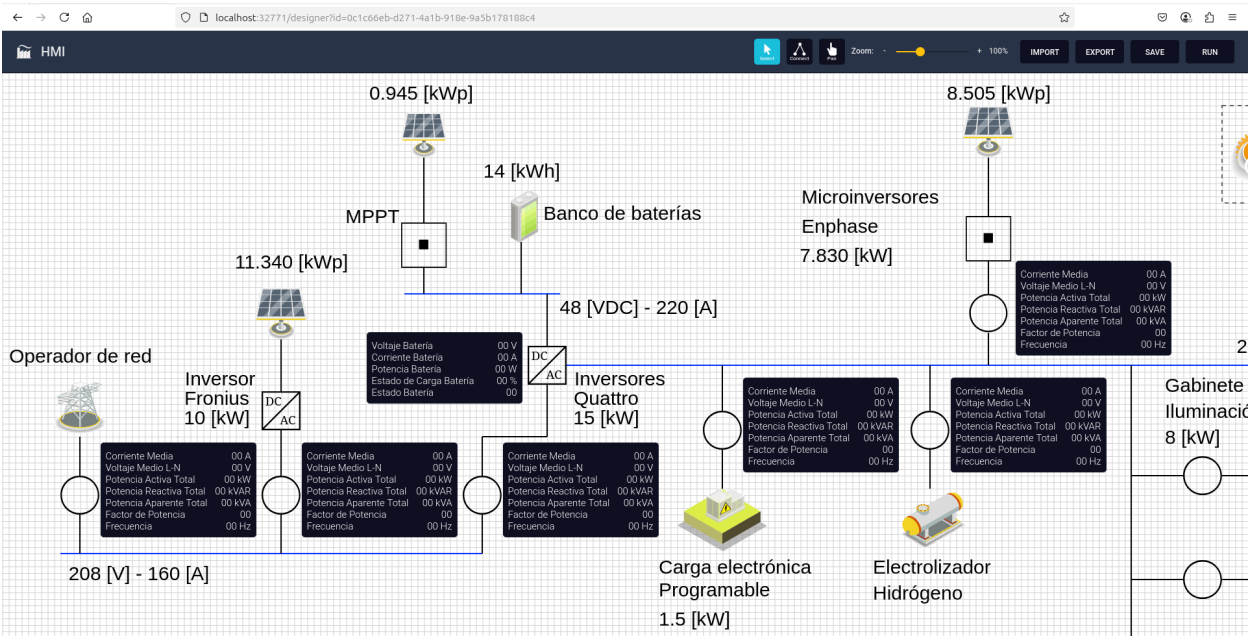


Figure 3: No auto zoom design

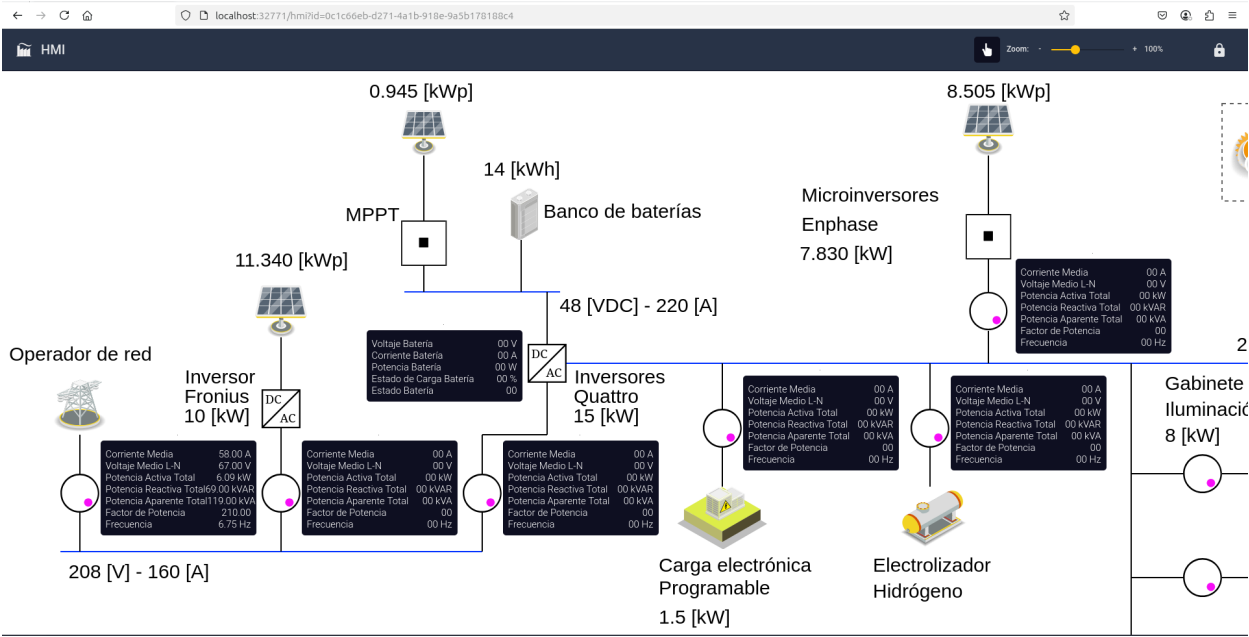


Figure 4: No auto zoom running

Now

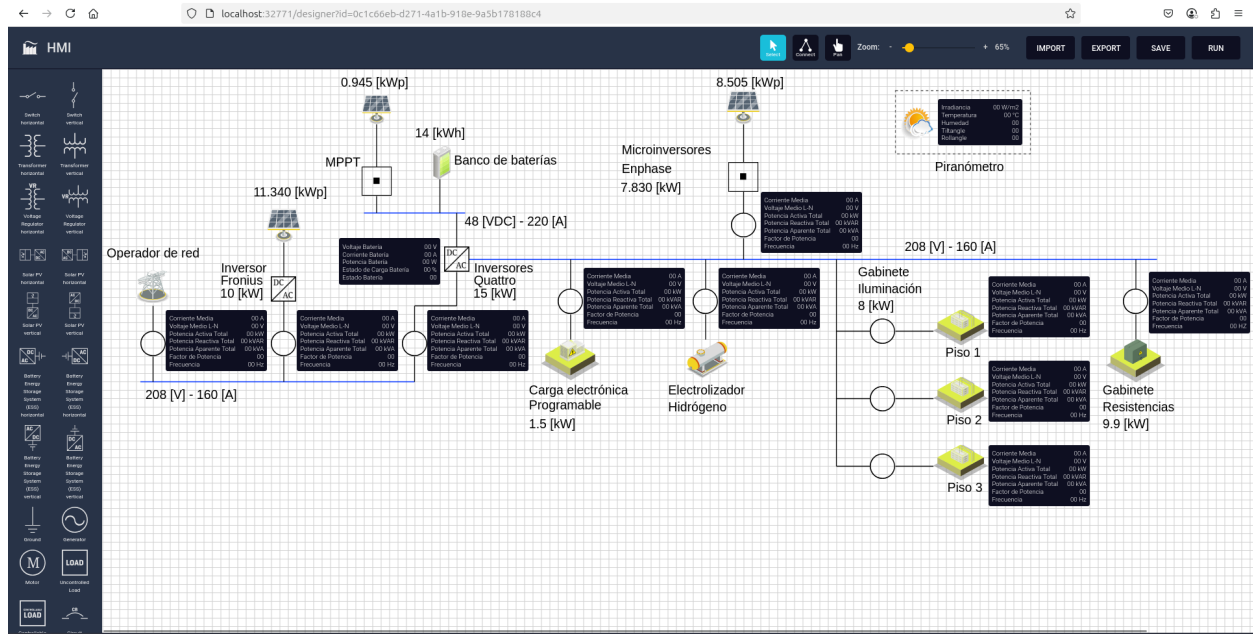


Figure 5: Auto zoom design

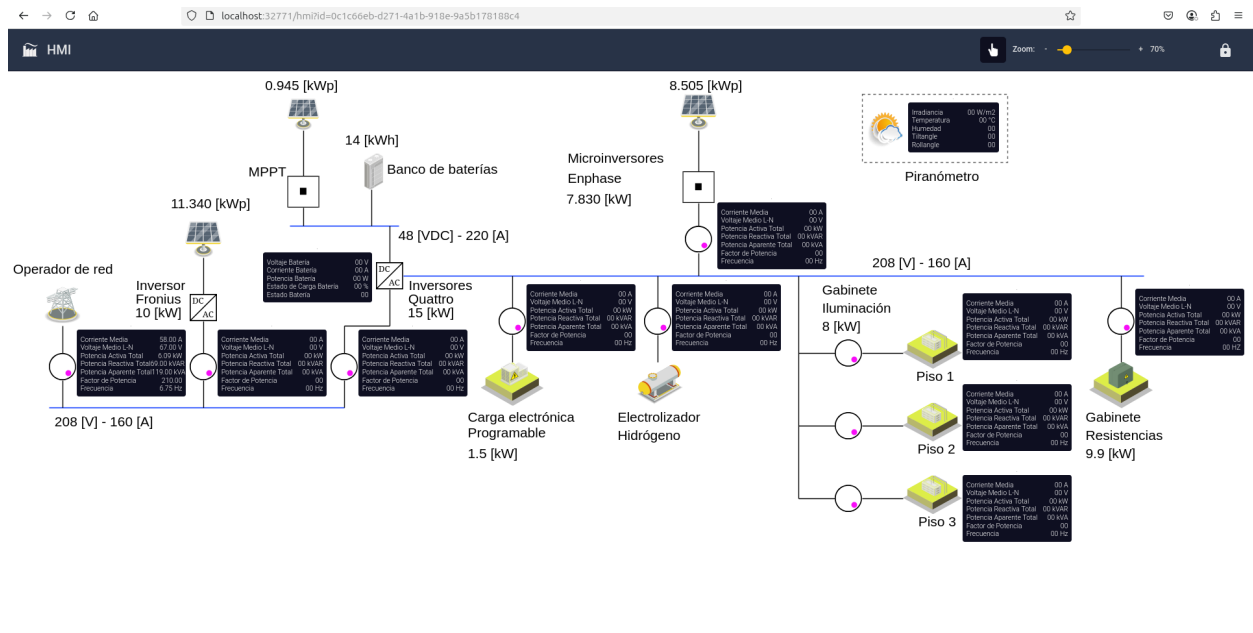


Figure 6: Auto zoom running

20.2 Overflow

Then

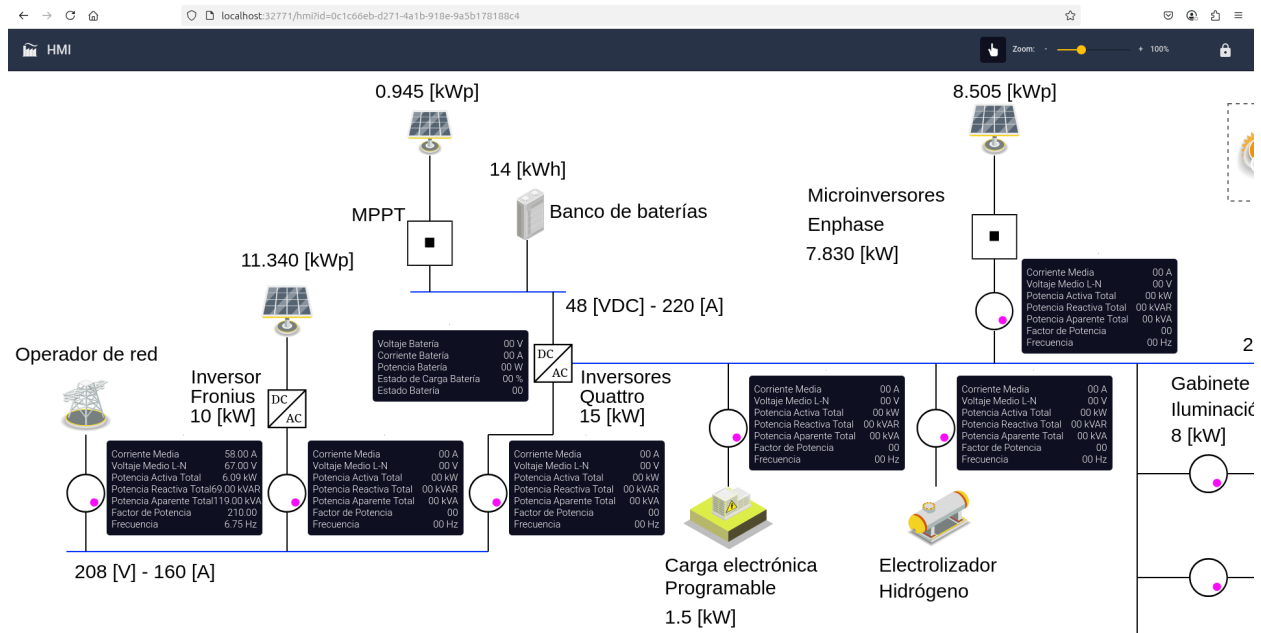


Figure 7: no overflow running

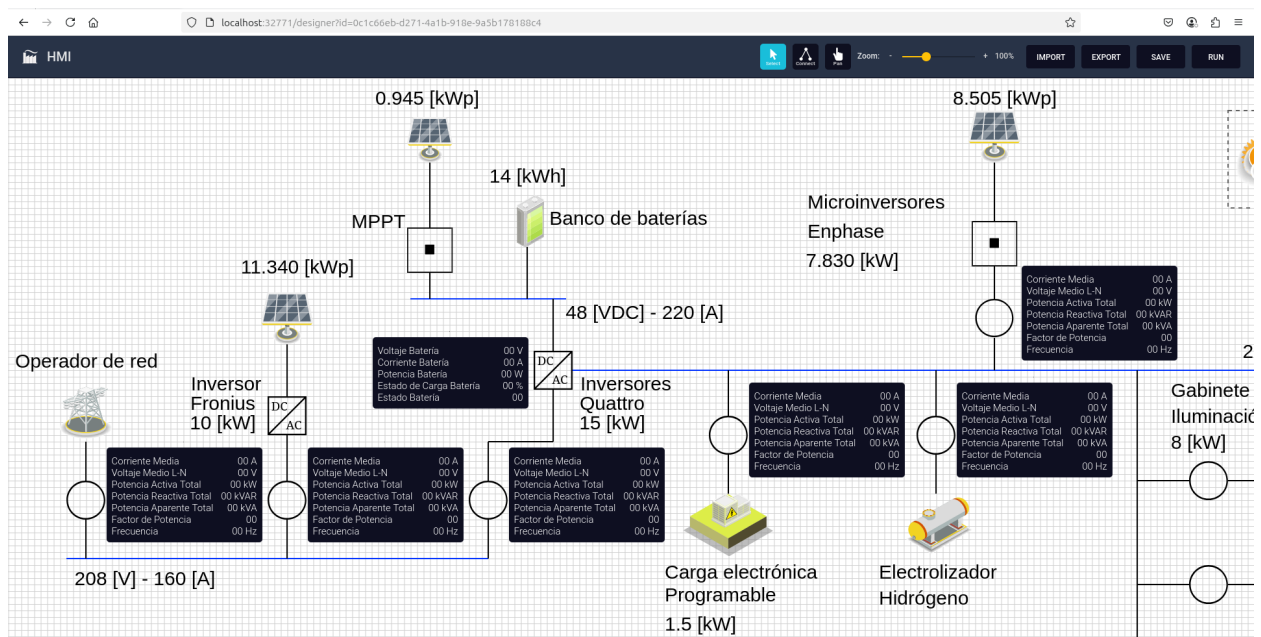


Figure 8: no overflow design

Now

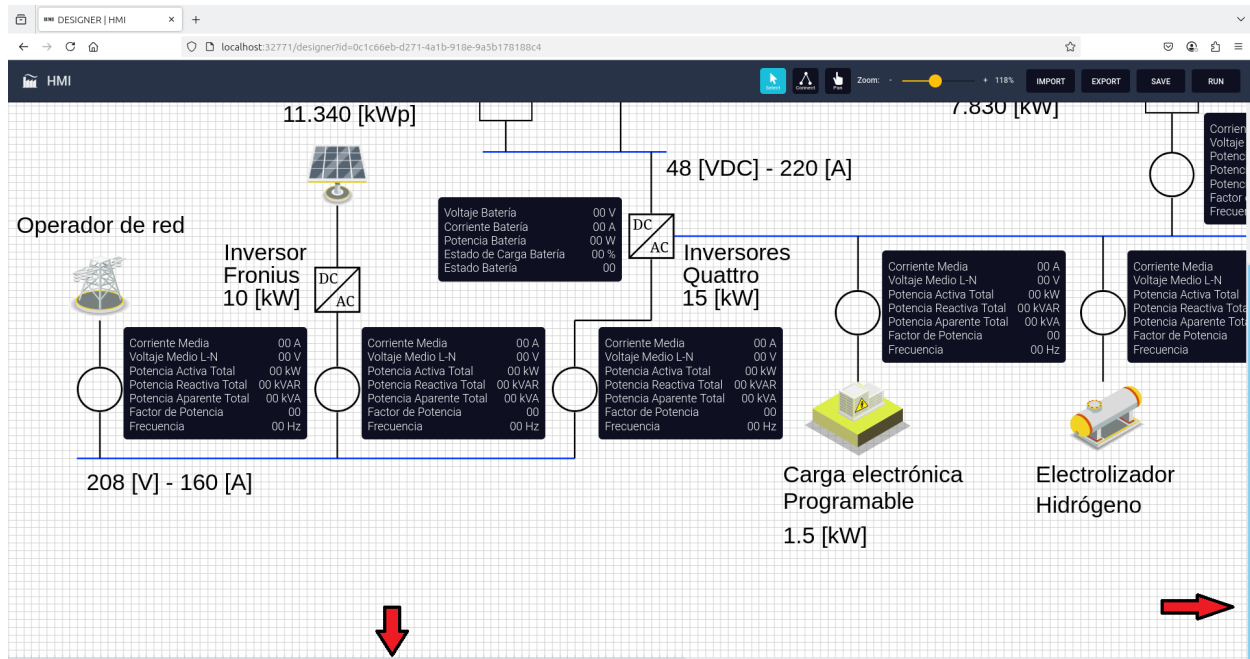


Figure 9: Overflow. Apply for running, too

20.3 Grafana Plotting Window

Then

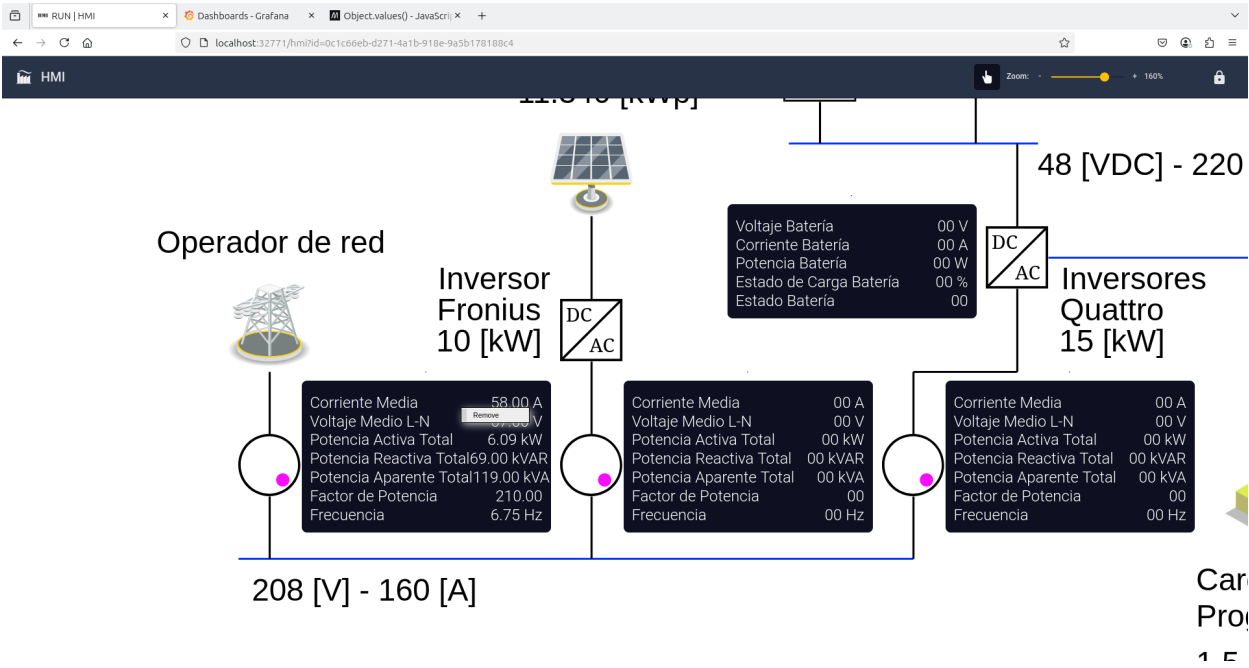


Figure 10: No Plotting Option

Now

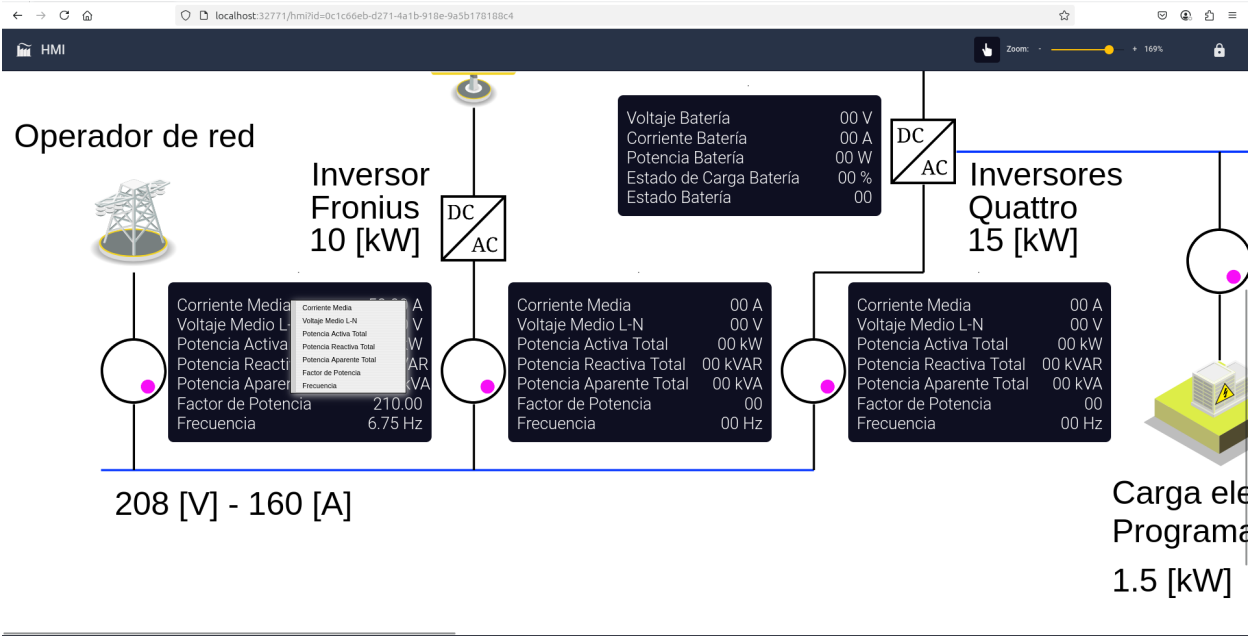


Figure 11: Plotting variables

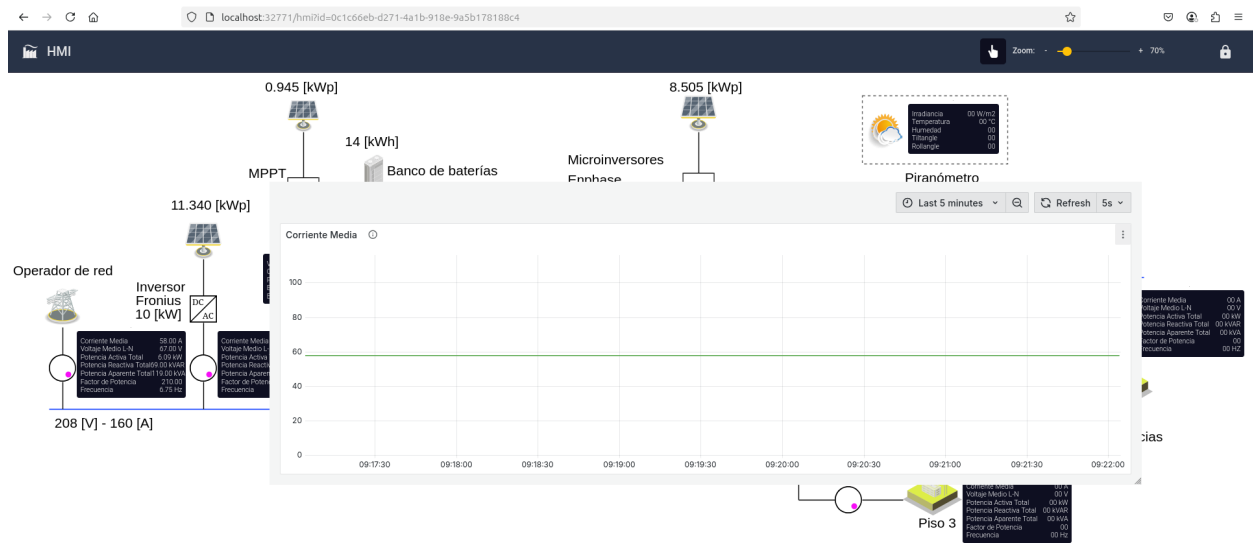


Figure 12: Grafana plot

20.4 Switches and Set-Points linking

Then

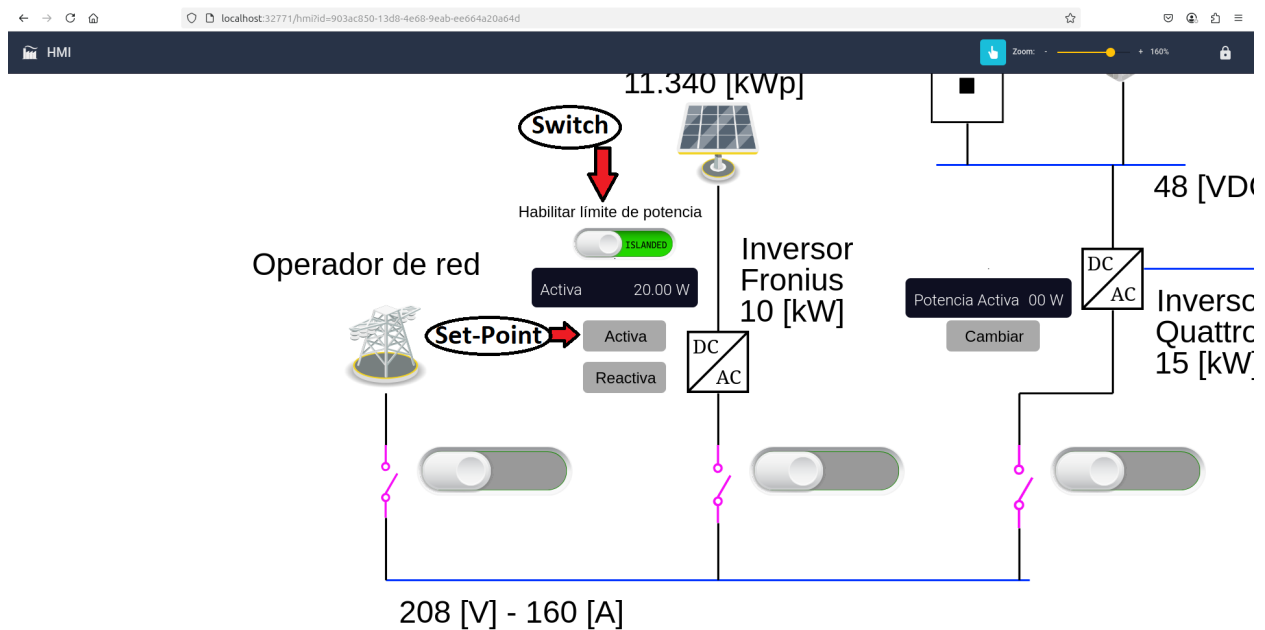


Figure 13: No link between the Switch and the Set-Point

Now. Note how the set-point changed its button color when the switch is in the isolated state, meaning that it is not possible to use it. Trying to double-click the button would not trigger any action, but once the switch is in the connected state, the set-point returns to its normal functionality.

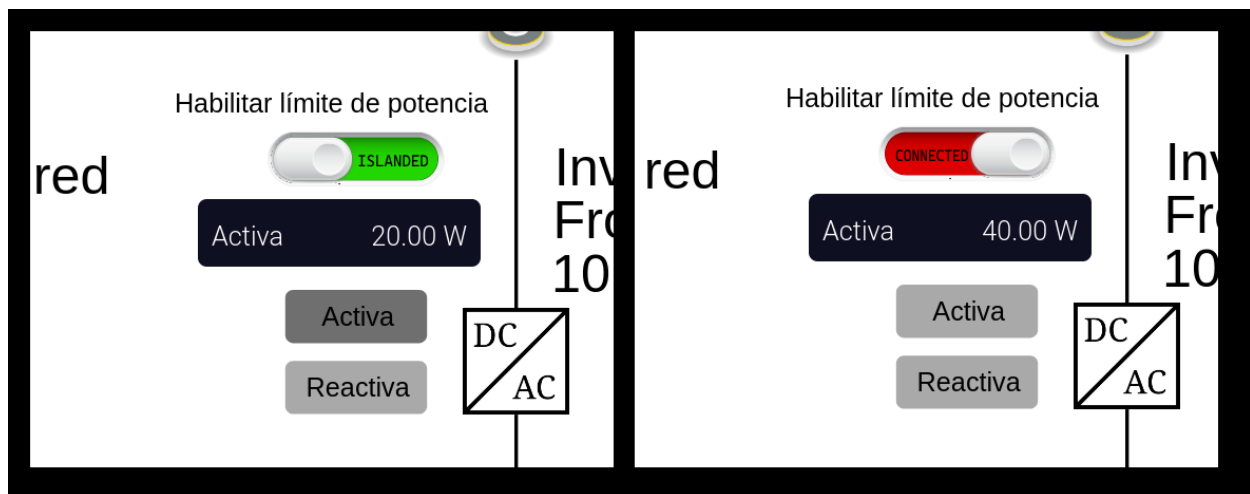


Figure 14: Link between the Switch and the Set-Point

20.5 Inserted Images

Custom SVG images are added to the designer toolbar and dragged onto diagrams. Images are registered in `toolbar.json` and copied into the Docker container at startup via the entrypoint script.

Now

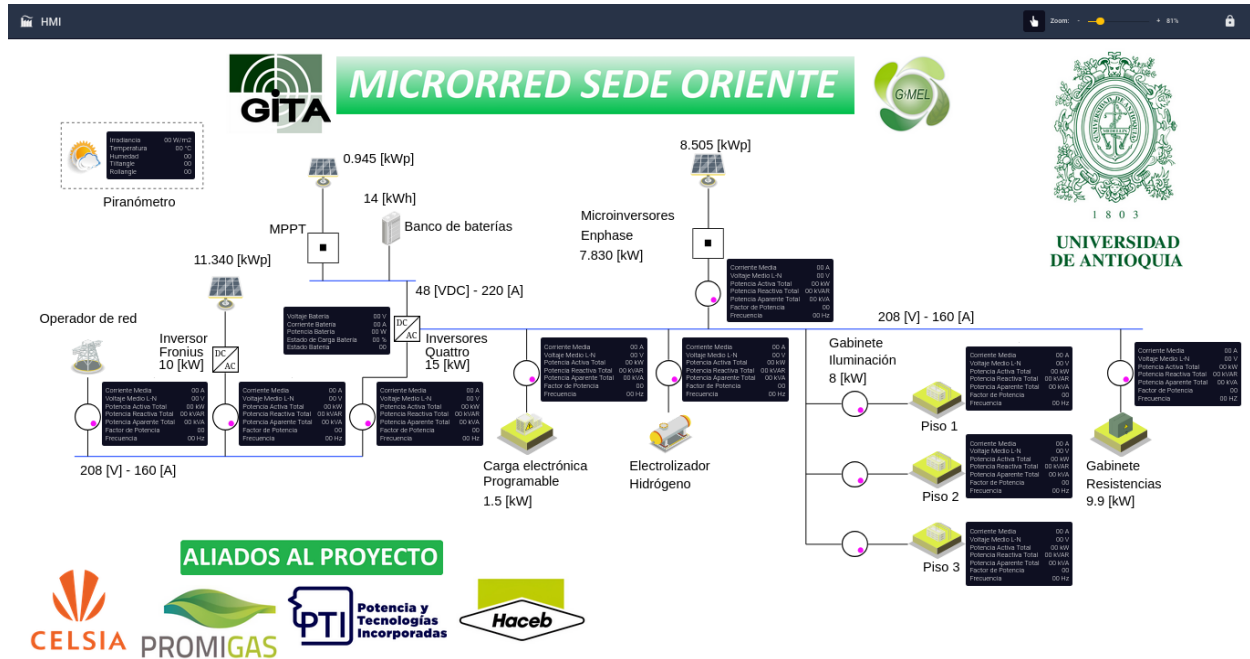


Figure 15: Design with images inserted one by one

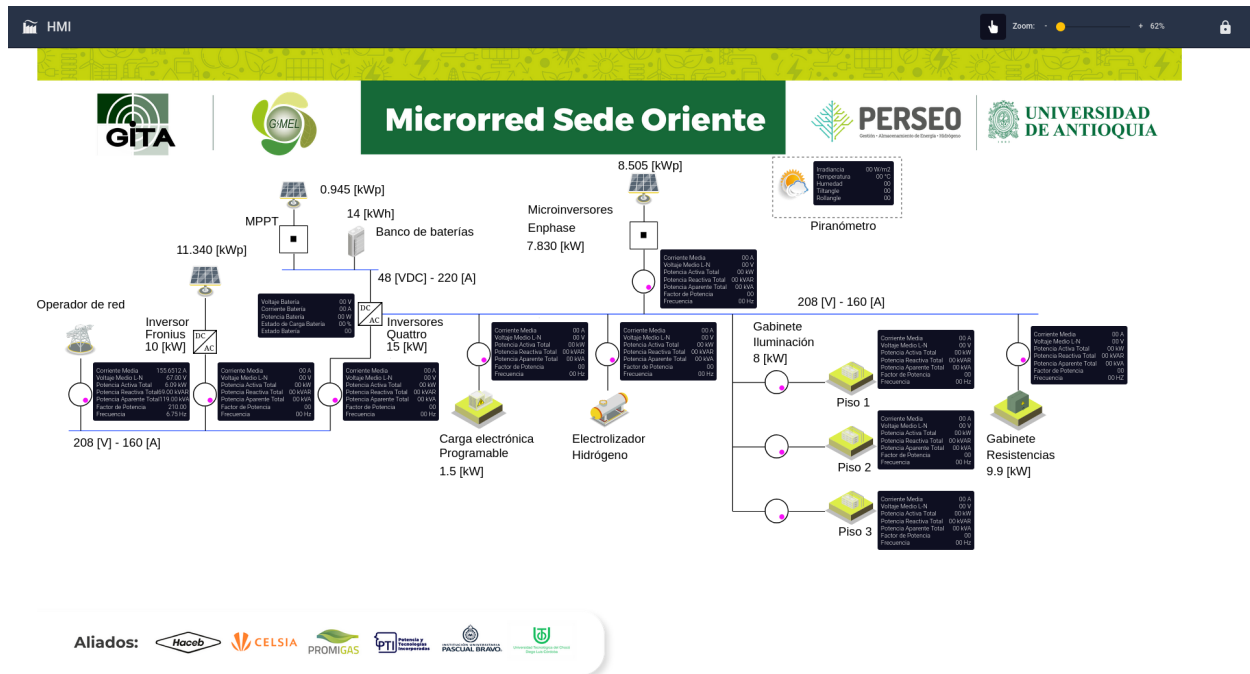


Figure 16: Design with one single image

21 Docker Commands

Frequent Docker commands:

- To run docker containers and recreated if necessary.
`$ docker compose up`
- To run docker containers without recreating.
`$ docker compose up --no-recreate`
- Show docker images.
`$ docker image ls`
- Show docker containers currently running.
`$ docker ps`
- Create a new docker image.
`$ docker build -t <name_image>:<version_identification> .`
- Push a docker image into Dockerhub. The Dockerhub account where this will be uploaded is the one signed in Docker desktop.
`$ docker push <name_image>:<version_identification>`

for Ubuntu, add "sudo" before the command to execute the context in admin mode. Pay attention that this context is different from the non-admin context. Containers running in admin mode would be seen in non-admin mode. In other words, `sudo docker ps != docker ps` based on the tests.

22 Git Commands

Frequent Git commands:

- Initialize a git repository.
`$ git init`
- Status of the git repository.
`$ git status`
- Add all files to track.
`$ git add .`
- Commit changes with their descriptions
`$ git commit -m "<message>" -m "<message>"`
- Creates a new Branch
`$ git branch <new branch name>`
- Create a new branch if not created and then check out.
`$ git checkout -b <branch name>`
- Show list of saved remote repositories.
`$ git remote -v`
- Add remote repository where pull requests or pushes would be requested.
`$ git remote add <repository name> <link>`
- Push the content of the current branch to a specific remote repository.
`$ git push -u <remote name>`
- Push the content of the current working branch to the last saved remote repository
`$ git push -u`
- Rebase local repo to make a copy of the remote repo
 1. `git fetch --prune` = Update the information about the remote repo and delete the not existing remote branches
 2. `git reset --hard <remote repo>/<remote branch>` = Update/Reset the content of the files to be the same as the remote branch. Also, resets the index and working tree. Any changes to tracked files in the working tree since `commit` are discarded. Any untracked files or directories in the way of writing any tracked files are simply deleted.
 3. `git clean -fd` = Remove untracked files from the working tree, searching recursively through untracked directories and forcing its modification or deletion.

Note: Some folders present at the moment the commands are executed may not be deleted.

23 Naming Convention

Most of the naming conventions applied to the code added are listed below:

- Prefixes
 - `p_` = Singular function parameter.
 - `param_` = Custom function-parameter type.
 - `enum_` = enumerator.
 - `names_` = Namespace.
 - `return_` = Return type.
 - `type_` = Custom typing variable.
 - `m_` = Module.
 - `api` = HTTP-related.
- Global constants are named on uppercase and snake_case with `GRAFANA_API_` as the prefix.
- Functions are named with camelCase.
- Classes with PascalCase.
- Local variables with camelCase.
- Files are named as: `GrafanaApi<purpose>Udea.ts`
- The main classes of any file are named with the `Grafana` prefix without the `_`. That is:
 - `GrafanaDatasource`
 - `GrafanaDashboard`
- The general formula for naming is:
`<prefix><purpose><extra>`
in this way:
 - `GrafanaDashboard`
 - `enum_errorKnownErrors`

24 Software Versions

In this section the versions of the main software, images or libraries are gathered together. Some specific versions for other libraries or software not explicitly installed, but implicitly installed alongside with the main ones, are supposed to work well when downloaded, but there is no guarantee. This content is for support for the user in troubleshooting.

24.1 Ubuntu

Lubuntu	Ubuntu 24.04.1 LTS
Docker Desktop	4.35.1
Docker Core	v27.3.1
ModbusPal	1.6c
Coretto openjdk	17.0.13.11.1
docker image oesinc/openfmb.adapters	a30d1d0
docker image nats	sha256: d15749dc10d8e67b55f496551ca3794b 2f131556342b313eb3e6115ec75f6fb3
docker image juangcjudaea/udea_perseo_p4	v2.2_test
docker image timescale/timescaledb	0
docker image grafana/grafana	11.5.1
docker image juangcjudaea/udea_perseo_p4_reverse_proxy	lastest
Inkscape	1.4
node	v18.19.1
npm	9.2.0
cmake	3.28.3 (kitware)
angular cli	15.1.0
rust	rustc 1.90.0
python	3.12.3

Table 2: Software Versions

24.2 Windows

Windows 10	10.0.19045.6216
WSL	2.5.10.0
Ubuntu Distro	24.04.1 LTS
Docker Desktop	4.37.1 (178610)
Docker Core	v27.4.0, build bde2b89
ModbusPal	1.6c
Coretto openjdk	26.0.6
docker image oesinc/openfmb.adapters	a30d1d0
docker image nats	sha256:58c362ade06c7e353be71dc630bc 201f59bca9e37126627aed51849b00bdfb1d
docker image juangcjudaea/udea_perseo_p4	v2.2_test
docker image timescale/timescaledb	sha256:645fd9e92d760673d49952710274 f88cc4771ec5623e2bdfc976c53c0211954c
docker image grafana/grafana	11.5.1
docker image juangcjudaea/udea_perseo_p4_reverse_proxy	lastest
Inkscape	1.4.2
node	v22.14.0
npm	10.9.2
cmake	3.30.7
angular cli	15.1.0
rust	1.90.0
python	3.13.2
Visual Studio Build Tools 2022	-

Table 3: Software Versions